Detecção automática de vagas de estacionamento por processamento digital de imagens clássico: calibração do limiar, benchmark na base PKLot e análise dos modos de falha

Automatic parking space detection using classical digital image processing: threshold calibration, PKLot benchmark and failure-mode analysis

Eduardo Morais Silva Martins*

Novembro de 2026

Resumo

Este trabalho propõe e avalia um sistema de detecção de vagas de estacionamento ocupadas e livres baseado exclusivamente em processamento digital de imagens (PDI) clássico, sem aprendizado de máquina. O pipeline, implementado em C++20 com OpenCV, aplica conversão para tons de cinza, filtragem gaussiana, limiarização adaptativa e dilatação morfológica sobre a imagem inteira e, em seguida, conta os pixels ativos dentro do polígono de cada vaga, decidindo a ocupação por um limiar τ. A calibração de τ no pátio UFPR04 da base PKLot indicou $\tau = 900$ (acurácia de 85,81% e F1-Score médio de 82,37%). Fixado esse valor, o sistema foi avaliado nas 12.416 imagens processadas da base, totalizando 719.528 decisões de vaga, com acurácia global de 74,89%, F1-Score de 70,47% e cerca de 21,2 quadros por segundo em CPU. O desempenho varia com a geometria da câmera: 87,42% de acurácia no UFPR04 (4º andar), 78,45% no UFPR05 e 70,59% na PUCPR (10º andar), onde o recall cai a 47,95% porque cada vaga ocupa poucos pixels e o limiar absoluto deixa de ser adequado. A análise dos erros relaciona os falsos positivos a sombras duras e reflexos em pavimento molhado e os falsos negativos à escala da vaga na imagem, e propõe normalizar o limiar pela área da vaga, suprimir sombras no espaço HSV e integrar decisões no tempo.

Palavras-chave: PKLot. vagas de estacionamento. processamento digital de imagens. limiarização adaptativa. OpenCV.

*Graduando em Engenharia de Computação, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), campus Nova Gameleira, Belo Horizonte, MG. Disciplina de Visão Computacional, Profª Drª Natália Batista.

Abstract

This paper presents and evaluates a parking space occupancy detector built exclusively on classical digital image processing, with no machine learning. A C++20/OpenCV pipeline applies grayscale conversion, Gaussian filtering, adaptive thresholding and morphological dilation to the whole frame, then counts active pixels inside each annotated space polygon and decides occupancy with a threshold τ. Calibrating τ on the UFPR04 lot of the PKLot dataset yielded $\tau = 900$ (85.81% accuracy, 82.37% mean F1-Score). With τ fixed, the system was evaluated on 12,416 images (719,528 space-level decisions), reaching 74.89% global accuracy, 70.47% F1-Score and about 21.2 frames per second on a CPU. Performance depends strongly on camera geometry: 87.42% accuracy on UFPR04 (4th floor), 78.45% on UFPR05 and 70.59% on PUCPR (10th floor), where recall drops to 47.95% because each space covers few pixels and an absolute threshold no longer fits. The error analysis links false positives to hard shadows and wet-pavement reflections and false negatives to space scale in the image, and proposes area-normalised thresholds, HSV shadow suppression and temporal integration.

Keywords: PKLot. parking spaces. digital image processing. adaptive thresholding. OpenCV.

1 Introdução

A busca por vagas em grandes estacionamentos consome tempo, combustível e atenção do motorista, e contribui para o congestionamento interno e para a emissão de poluentes. As soluções comerciais se dividem em sistemas de contagem nas entradas e saídas, que informam o total de vagas livres mas não a sua posição, e sistemas com sensores por vaga, como os ultrassônicos e os magnéticos (WOLFF et al., 2006), cujo custo de instalação e manutenção cresce com o número de vagas. Uma alternativa é reaproveitar câmeras de vigilância (CCTV) já instaladas: uma única câmera enquadra dezenas ou centenas de vagas (ALMEIDA et al., 2015).

O estado da arte recente em ocupação de vagas usa redes neurais convolucionais (AMATO et al., 2017), que são precisas, mas exigem dados anotados, treinamento e hardware com maior capacidade de cálculo. Este trabalho adota deliberadamente o caminho oposto: um pipeline de PDI clássico e decisão por limiar, justificado por quatro razões de engenharia: (i) custo computacional baixo, compatível com *edge computing* em CPU; (ii) tempo de execução previsível, com complexidade linear no número de pixels; (iii) explicabilidade *white-box*, pois cada decisão decorre de propriedades observáveis da imagem, o que permite localizar a causa de cada erro; e (iv) ausência de treinamento, e portanto de sobreajuste a uma câmera específica.

O objetivo é construir esse sistema, avaliá-lo na base PKLot e, principalmente, entender onde e por que ele falha. Não se busca superar métodos supervisionados, mas medir os limites de uma formulação simples e interpretável. As contribuições são: (a) um pipeline modular em C++20 com mascaramento poligonal de cada vaga; (b) a calibração empírica do limiar τ; (c) um benchmark sobre 719.528 decisões de vaga em três pátios e três condições climáticas; e (d) uma análise dos modos de falha ligada à física da formação da imagem, com a identificação do efeito da altura da câmera sobre o limiar.

2 Referencial teórico

Tons de cinza. A imagem colorida é reduzida à luminância $Y = 0{,}299R + 0{,}587G + 0{,}114B$ (GONZALEZ; WOODS, 2010), o que descarta a crominância e deixa o método dependente da variação de intensidade, e não da cor da lataria.

Filtragem gaussiana. O ruído de sensor e de compressão é atenuado pela convolução com o núcleo

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{1}$$

que é um passa-baixas isotrópico e separável (GONZALEZ; WOODS, 2010).

Limiarização adaptativa. Em vez de um limiar global, cada pixel é comparado à média ponderada da sua vizinhança (BRADLEY; ROTH, 2007). Com janela gaussiana w de 25×25 e constante C, o limiar local e a imagem binária (variante invertida, `THRESH_BINARY_INV`) são

$$T(x, y) = \sum_{(u,v)} w(u, v)\, I_s(x + u, y + v) - C, \qquad B(x, y) = \begin{cases} 255, & I_s(x, y) < T(x, y) \\ 0, & \text{caso contrário.} \end{cases} \tag{2}$$

Assim, um pixel é ativado quando é mais escuro que a sua vizinhança em mais de C níveis de cinza. Isso torna o método tolerante a variações lentas de iluminação, mas sensível a qualquer descontinuidade local de brilho, inclusive a borda de uma sombra.

Morfologia matemática. A dilatação por um elemento estruturante S, $(B \oplus S)(x, y) = \max_{(s,t)\in S} B(x - s, y - t)$, une fragmentos próximos e conecta as arestas de um mesmo objeto (HARALICK; STERNBERG; ZHUANG, 1987).

Característica por densidade. Para a vaga i, com região poligonal P_i, a característica é a contagem de pixels ativos e a decisão é por limiar:

$$p_i = \sum_{(x,y)\in P_i} \mathbb{1}[D(x, y) > 0], \qquad \hat{y}_i = \mathbb{1}[p_i \geq \tau], \tag{3}$$

em que D é a imagem dilatada e $\hat{y}_i = 1$ indica vaga ocupada. A hipótese é que um veículo, com para-brisa, vidros, rodas e vincos de lataria, produz muitas transições de brilho, e o asfalto liso produz poucas.

Métricas. Com verdadeiros positivos (VP), verdadeiros negativos (VN), falsos positivos (FP, vaga livre classificada como ocupada) e falsos negativos (FN, carro não detectado), definem-se acurácia $= (VP + VN)/N$, precisão $= VP/(VP + FP)$, *recall* $= VP/(VP + FN)$ e F1 $= 2PR/(P + R)$.

3 Trabalhos relacionados

Os sistemas baseados em imagem dividem-se em orientados a carro, que detectam veículos, e orientados a vaga, que classificam cada espaço (ALMEIDA et al., 2015). Os primeiros sofrem com a perspectiva, pois um carro distante tem poucos pixels. Entre os segundos, a subtração de fundo assume um fundo estatisticamente estacionário, hipótese que falha ao ar livre. Sastre et al. (SASTRE et al., 2007) usaram filtros de Gabor, Ichihashi et al. (ICHIHASHI et al., 2009) usaram PCA com classificador fuzzy e Huang e Wang (HUANG; WANG, 2010) propuseram um modelo bayesiano hierárquico que considera a

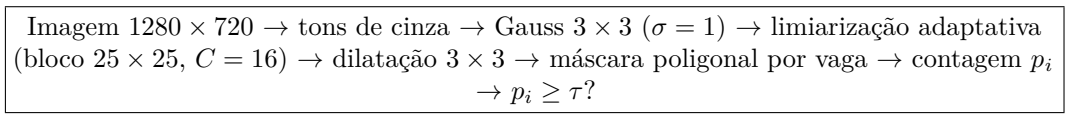
oclusão entre vagas. Funck et al. (FUNCK; MöHLER; OERTEL, 2004) já determinavam a ocupação a partir de imagens únicas.

O trabalho que fundamenta este estudo é o de Almeida et al. (ALMEIDA et al., 2015), que apresentam a base PKLot (12.417 imagens, 695.899 recortes de vagas) e um *baseline* com descritores de textura (LBP e LPQ) e SVM. Os autores obtêm mais de 99% de acurácia quando treino e teste usam o mesmo pátio, mas cerca de 89% como melhor resultado quando o classificador é aplicado a um pátio não visto, o que mostra a dificuldade de generalizar entre câmeras. Esse protocolo usa classificadores treinados e divisão treino/teste por dia, enquanto o método aqui proposto não treina nada, avalia todas as imagens e usa um único limiar, de modo que os números não são diretamente comparáveis. A posição deste trabalho é a de uma alternativa determinística e de baixo custo, cujo comportamento de erro é analisado em profundidade, e não a de um concorrente em acurácia. O resultado entre pátios (Seção 5.3) mostra a mesma tendência de queda de desempenho entre câmeras relatada pelos autores, porém com magnitude maior na PUCPR.

4 Metodologia proposta

O sistema foi implementado em C++20, com OpenCV (BRADSKI, 2000) para PDI, `pugixml` (KAPOULKINE, 2024) para ler os XMLs de anotação e CMake como sistema de construção. Há três módulos: `DatasetParser` (anotações), `ParkingPipeline` (PDI e decisão) e `Evaluator` (métricas e registro em CSV). A Figura 1 resume o fluxo.

Figura 1 – Pipeline proposto: PDI global sobre o quadro e decisão por vaga.



Fonte: Elaborado pelo autor.

PDI global. A conversão para cinza, o filtro gaussiano 3×3 com $\sigma = 1$, a limiarização adaptativa gaussiana invertida (bloco 25×25, $C = 16$) e a dilatação com elemento retangular 3×3 são aplicados uma única vez ao quadro completo. Isso evita repetir operações por vaga e preserva, na binarização, o contexto de vizinhança que seria perdido ao recortar a vaga antes. Em `adaptiveThreshold`, o OpenCV deriva o σ da janela gaussiana de 25×25 como $0{,}3\,((25-1)/2-1)+0{,}8 = 4{,}1$.

Mascaramento por vaga. Para cada vaga, lê-se do XML o polígono de quatro vértices, calcula-se a caixa delimitadora, recorta-se essa região da imagem binária e preenche-se uma máscara convexa (`fillConvexPoly`) com os vértices exatos. A operação `bitwise_and` zera o que está fora do polígono, de modo que faixas, calçadas e carros vizinhos não contribuem para a contagem. A caixa delimitadora é recortada pelos limites do quadro antes do uso.

Decisão. A característica p_i é obtida por `countNonZero` sobre a região mascarada, e a vaga é classificada como ocupada se $p_i \geq \tau$. O limiar τ é o único hiperparâmetro de decisão e é calibrado na Seção 5.1. Os rótulos (`occupied` $\in \{0, 1\}$) vêm do mesmo XML e servem apenas para a avaliação.

Protocolo. O método não tem etapa de treinamento, de modo que não há divisão

treino/teste: τ é ajustado em um subconjunto de UFPR04 e depois fixado para todos os pátios. A condição *Overcast* da publicação original corresponde à pasta `Cloudy` usada no código e é denominada "nublado" neste texto. A contagem de FPS inclui a leitura do JPEG, o *parsing* do XML, o PDI e a decisão, ou seja, é uma medida ponta a ponta.

5 Resultados experimentais e análise de falhas

5.1 Calibração do limiar τ no pátio UFPR04

O limiar foi varrido de 700 a 2.500 pixels (passo de 100) no UFPR04, com 200 imagens por condição climática, o que dá 5.600 vagas por clima e 16.800 avaliações por rodada. A Tabela 1 mostra os pontos mais informativos.

Tabela 1 – Calibração de τ no UFPR04 (200 imagens por clima; 16.800 avaliações por valor de τ).

τ	Acur. Sunny	Acur. Cloudy	Acur. Rainy	**Acur. global**	**F1 médio**
700	69,29%	89,55%	79,71%	79,52%	77,32%
800	77,88%	89,11%	85,70%	84,23%	81,09%
900	82,38%	86,43%	88,62%	**85,81%**	**82,37%**
1000	83,95%	81,00%	89,11%	84,68%	80,92%
1100	82,48%	74,46%	88,39%	81,78%	76,93%
1500	76,27%	48,07%	83,59%	69,31%	54,76%
2500	70,64%	29,66%	78,38%	59,56%	22,36%

Fonte: Elaborado pelo autor, a partir de `historico_experimentos.csv`.

O comportamento é o de um compromisso entre dois erros. Para $\tau < 900$ o *recall* é alto (cerca de 96%), mas a precisão cai nas condições de sol e chuva (cerca de 53% e 54%): sombras de árvores e reflexos em poças geram entre 700 e 850 pixels ativos em vagas livres, o que produz mais de 1.600 FP. Para $\tau > 1000$ o sistema fica restritivo demais: sob luz difusa os veículos geram contraste menor, em torno de 800 a 1.100 pixels, e os FN passam a 1.320 em $\tau = 1100$ e a 2.848 em $\tau = 1500$. O melhor equilíbrio ocorre em $\tau = 900$, com acurácia global de 85,81%, F1 médio de 82,37% e *recall* de 86,99%. Como a condição nublada degrada primeiro com o aumento de τ e a ensolarada só melhora até $\tau = 1000$, nenhum limiar único é ótimo para as três condições, e o valor 900 prioriza o ganho em precisão no sol e na chuva sem perda grande de *recall* no nublado. Esse subconjunto de calibração é uma parte do próprio UFPR04, de modo que os resultados desse pátio no benchmark são, em alguma medida, otimistas.

5.2 Benchmark em larga escala

Com $\tau = 900$ fixo, foram processadas 12.416 imagens da base (a publicação original informa 12.417; a diferença é uma imagem em PUCPR/Sunny, 2.314 processadas contra 2.315), com 719.528 decisões de vaga. A Tabela 2 apresenta os resultados por pátio e condição, e a Tabela 3 os consolida por pátio.

O resultado global, de 74,89% de acurácia e 70,47% de F1, esconde uma heterogeneidade grande (Figura 2 e Tabela 3). O pátio PUCPR responde por 62,2% das decisões avaliadas (447.300 de 719.528) e, portanto, puxa a média global para baixo. A matriz de confusão agregada (Figura 4) mostra que 120.145 dos erros são FN e 60.532 são FP,

Tabela 2 – Resultados do método de PDI clássico ($\tau = 900$) sobre as imagens processadas da base PKLot.

Pátio	Condição	Imagens	Vagas	Acurácia	Precisão	Recall	F1	FPS
UFPR04	Ensolarado	2.098	58.744	86,63%	84,73%	92,19%	88,31%	21,5
UFPR04	Nublado	1.408	39.424	88,31%	74,51%	91,67%	82,21%	21,9
UFPR04	Chuvoso	285	7.980	88,90%	76,94%	89,04%	82,55%	22,2
UFPR05	Ensolarado	2.500	100.000	78,39%	79,75%	83,73%	81,69%	21,4
UFPR05	Nublado	1.426	57.040	79,00%	81,48%	83,51%	82,48%	21,7
UFPR05	Chuvoso	226	9.040	75,56%	85,45%	76,72%	80,85%	21,7
PUCPR	Ensolarado	2.314	231.400	69,78%	70,81%	44,55%	54,69%	20,5
PUCPR	Nublado	1.328	132.800	77,53%	69,50%	52,66%	59,92%	20,7
PUCPR	Chuvoso	831	83.100	61,80%	86,59%	50,16%	63,52%	20,5
Total	**Todas**	**12.416**	**719.528**	**74,89%**	**78,08%**	**64,22%**	**70,47%**	**21,2**

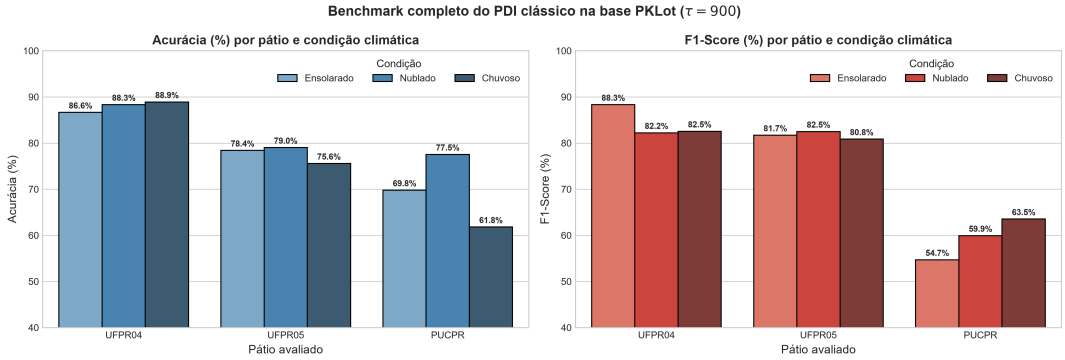
Fonte: Elaborado pelo autor, a partir de `historico_final_tau_900.csv`.

Tabela 3 – Resultados consolidados por pátio ($\tau = 900$).

Pátio	Vagas/imagem	Acurácia	Precisão	Recall	F1	FP	FN
UFPR04	28	87,42%	81,52%	91,90%	86,40%	9.613	3.736
UFPR05	40	78,45%	80,66%	83,21%	81,92%	19.443	16.353
PUCPR	100	70,59%	74,54%	47,95%	58,36%	31.476	100.056

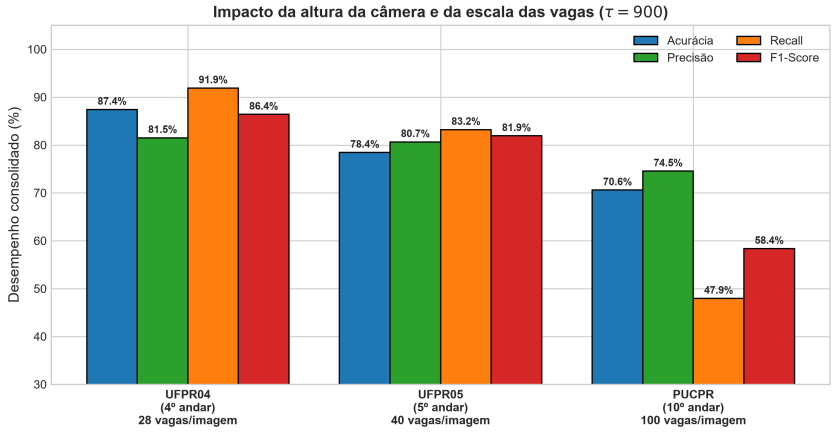
Fonte: Elaborado pelo autor. Vagas por imagem calculadas como vagas avaliadas divididas por imagens processadas.

Figura 2 – Acurácia e F1-Score por pátio e condição climática ($\tau = 900$).



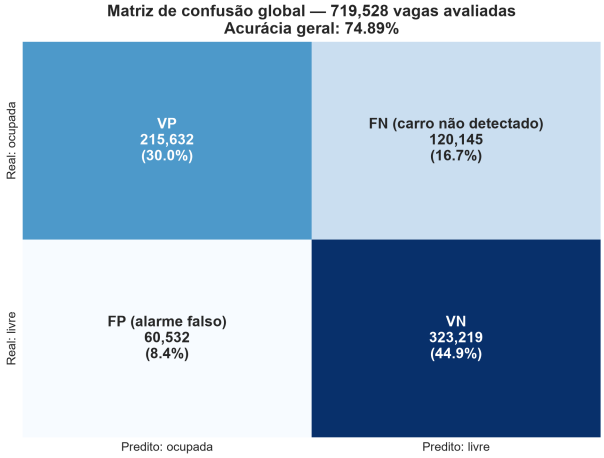
Fonte: Elaborado pelo autor.

Figura 3 – Impacto da altura da câmera e da resolução das vagas nas métricas consolidadas por pátio.



Fonte: Elaborado pelo autor.

Figura 4 – Matriz de confusão global agregada (719.528 decisões de vaga).



Fonte: Elaborado pelo autor.

ou seja, o sistema erra mais por não ver carros do que por inventá-los, e 83,3% dos FN (100.056) ocorrem na PUCPR.

Dois cuidados de leitura são necessários. Primeiro, no UFPR04 a condição nublada tem a maior acurácia (88,31%) e, ao mesmo tempo, uma precisão baixa (74,51%): isso se deve à prevalência, pois apenas 29,5% das vagas estão ocupadas, e então poucos FP por vaga livre pesam muito na precisão. Segundo, os resultados do UFPR04 no benchmark completo (por exemplo, 86,63% de acurácia em Sunny) diferem dos da calibração (82,38%) porque o subconjunto de 200 imagens e o conjunto completo têm composição diferente.

5.3 Comparação com o *baseline* do PKLot

O *baseline* de Almeida et al. (2015) não é diretamente comparável ao método aqui proposto: ele treina SVMs sobre descritores de textura (LBP e LPQ), usa metade dos dias

para treino e metade para teste e classifica recortes de vagas, ao passo que este trabalho não tem etapa de treinamento e avalia todas as imagens com um único limiar calibrado no UFPR04. O cenário mais próximo é o de treinar (aqui, calibrar) no UFPR04 e testar nos demais pátios, resumido na Tabela 4.

Tabela 4 – Acurácia deste trabalho ($\tau = 900$, calibrado no UFPR04, sem treinamento) e do *baseline* do PKLot (SVM com LBP/LPQ treinado no UFPR04).

Pátio de teste	**Baseline (fusão)**	**Baseline (melhor único)**	**Este trabalho**	**Diferença (fusão)**
UFPR04 (mesmo pátio)	99,64%	99,55%	87,42%	−12,22 p.p.
UFPR05	88,33%	84,92%	78,45%	−9,88 p.p.
PUCPR	88,40%	84,25%	70,59%	−17,81 p.p.

Fonte: Elaborado pelo autor, a partir da Tabela 3 de Almeida et al. (2015) e da Tabela 3.

O *baseline* é superior em todos os casos. No próprio UFPR04, o erro do método proposto é de 12,58%, contra 0,36% do *baseline*. Ao trocar de pátio, a queda de desempenho em relação ao UFPR04 é de 8,97 e 16,83 pontos percentuais (UFPR05 e PUCPR) neste trabalho, contra cerca de 11,3 e 11,2 pontos no *baseline*: a transferência para o UFPR05 degrada menos aqui, mas a transferência para a PUCPR degrada mais. A tendência qualitativa coincide com a relatada pelos autores, que atribuem a pior generalização para a PUCPR às diferenças de padrão do solo, ângulo de visão e altura de montagem da câmera (ALMEIDA et al., 2015), os mesmos fatores discutidos na Seção 5.5. Mesmo no *baseline*, o classificador treinado no UFPR04 rotula como vazias entre 19,3% e 27,4% das vagas ocupadas da PUCPR (Tabela 3(c) do artigo), contra 52,1% de carros não detectados neste trabalho (100.056 de 192.216), o que é compatível com a sensibilidade de um limiar absoluto à escala da vaga. Os erros que os autores analisam, como oclusão por carros vizinhos e árvores e alterações da superfície por sombra e chuva, correspondem em parte aos modos de falha identificados aqui. A vantagem do método proposto não está, portanto, na exatidão, e sim no custo computacional, na ausência de treinamento e na explicabilidade dos erros.

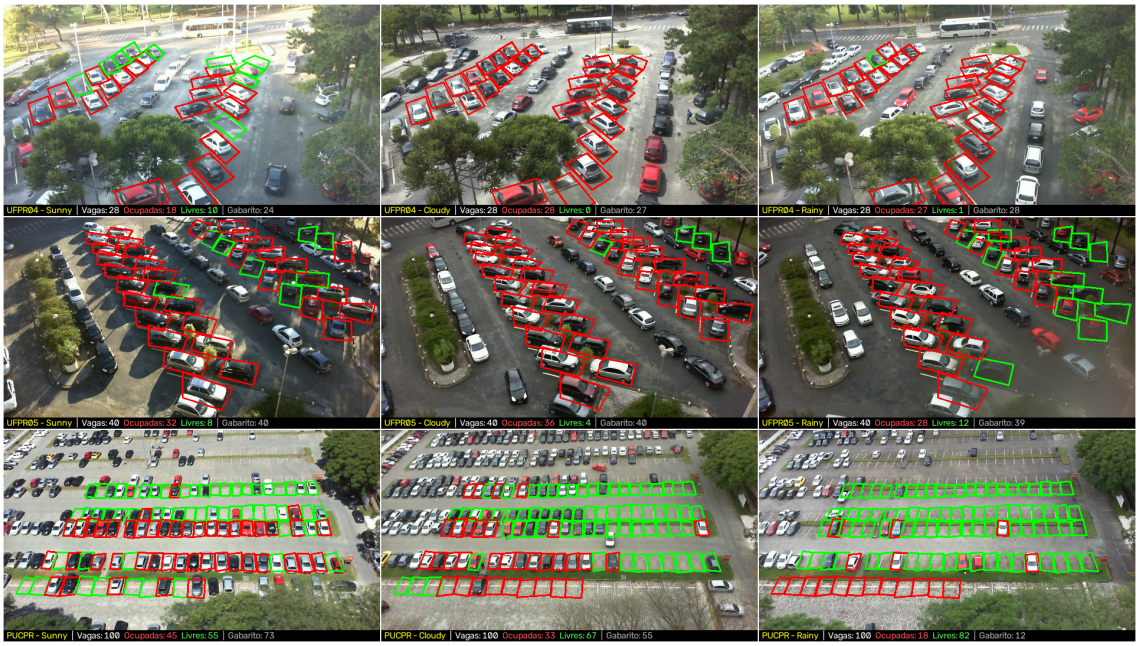
5.4 Exemplos qualitativos

Para complementar as métricas agregadas, a Figura 5 mostra uma imagem de cada combinação de pátio e condição climática processada com $\tau = 900$: as caixas vermelhas indicam vagas classificadas como ocupadas, as verdes como livres, e a faixa inferior de cada painel informa o total de vagas, as contagens preditas de ocupadas e livres e, para comparação, o número de vagas ocupadas segundo o gabarito. A Figura 6 apresenta as mesmas nove imagens após a binarização e a dilatação, isto é, a representação sobre a qual p_i é contado. A seleção é ilustrativa (uma amostra por combinação) e não substitui as estatísticas das Tabelas 2 e 3.

5.5 Análise dos modos de falha

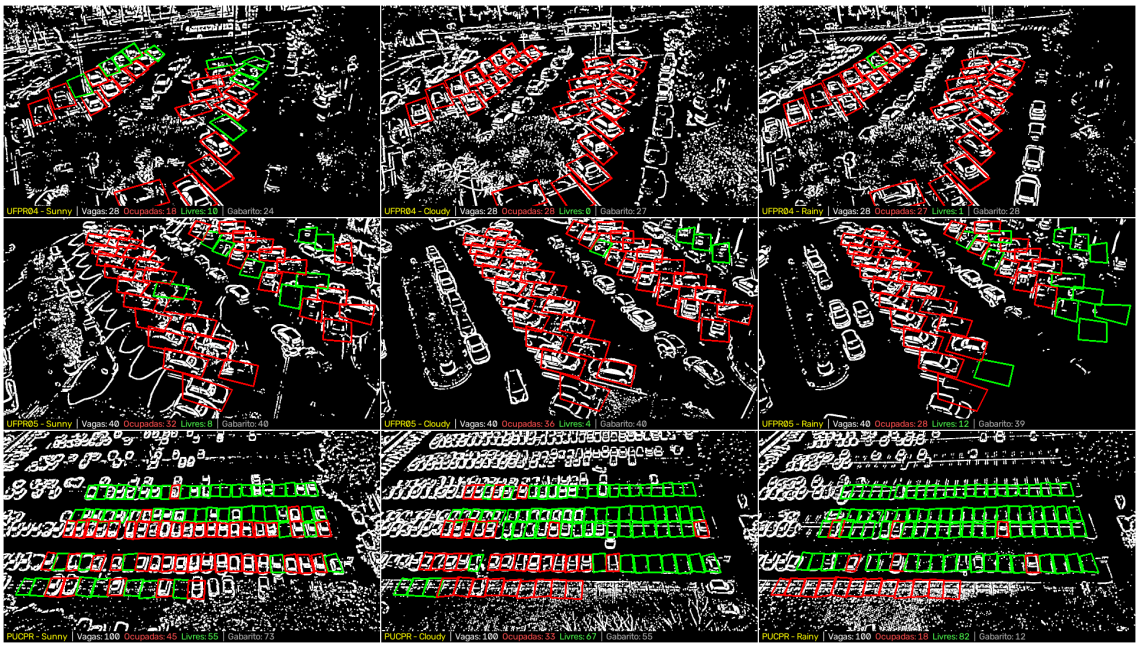
Sombras projetadas no sol (FP). Em dias claros, copas de árvores, postes e carros vizinhos projetam sombras de borda nítida sobre vagas vazias. Como a limiarização adaptativa ativa pixels mais escuros que a média local em mais de $C = 16$ níveis, a transição abrupta entre asfalto iluminado e sombreado gera uma faixa ativa de intensidade e forma semelhantes à borda de um veículo. A dilatação conecta essa faixa e eleva p_i acima de τ.

Figura 5 – Amostras de cada pátio (linhas) e condição climática (colunas) com a classificação por vaga ($\tau = 900$): vermelho = ocupada; verde = livre.



Fonte: Elaborado pelo autor.

Figura 6 – Imagens binarizadas (limiarização adaptativa e dilatação) das mesmas amostras da Figura 5, com os polígonos das vagas.



Fonte: Elaborado pelo autor.

Isso explica a precisão limitada em Sunny no UFPR05 (79,75%) e o grande número de FP naquele pátio (19.443).

Reflexos em pavimento molhado (FP). A lâmina d'água se comporta como superfície especular: pela refletância de Fresnel da interface ar–água, que cresce com a incidência rasante, poças refletem o céu e as luminárias, e o impacto das gotas adiciona ruído granular de alta frequência. A binarização trata esse padrão como estrutura, e a dilatação o agrega em blocos que superam τ. A chuva também reduz a luz disponível e o contraste da lataria, o que afeta ao mesmo tempo os dois lados do limiar. A condição chuvosa ocupa poucas imagens (285, 226 e 831), de modo que as estimativas para ela têm maior incerteza.

Textura do piso (FP). Nas imagens binarizadas da PUCPR (Figura 6), observa-se que o piso de bloquete, sob iluminação fraca, produz uma nuvem de pixels ativos de alta frequência, que para a contagem p_i se assemelha à estrutura de um veículo e pode elevar p_i em vagas livres. Trata-se de um indício qualitativo, ainda não quantificado por pátio; Almeida et al. (2015) também apontam o padrão do solo entre os fatores que distinguem a PUCPR dos demais pátios.

Altura da câmera e escala da vaga (FN). A PUCPR é filmada do 10º andar e enquadra 100 vagas em 1280×720, contra 28 vagas no UFPR04 (4º andar) e 40 vagas por imagem no UFPR05. Com a mesma resolução da matriz, a área em pixels de cada vaga decresce com a distância e com o número de vagas enquadradas. Estima-se que uma vaga da PUCPR ocupe menos de um terço da área em pixels de uma do UFPR04. Como p_i é uma contagem absoluta, um carro na PUCPR raramente atinge 900 pixels ativos, mesmo com todas as suas arestas corretamente segmentadas. O resultado é a subdetecção sistemática: *recall* de 44,55% a 52,66% nas três condições e 100.056 carros não vistos, enquanto a precisão se mantém entre 69,5% e 86,6%. O padrão de FN alto com precisão preservada é a assinatura de um limiar alto demais para a escala, e não de um problema de segmentação. Os pátios seguem uma ordem monótona com a geometria (acurácia de 87,42%, 78,45% e 70,59%, com *recall* de 91,90%, 83,21% e 47,95%, para UFPR04, UFPR05 e PUCPR). Esse é o achado central: τ não é invariante à escala da cena, e a sua transferência entre câmeras exige $\tau_i \propto A_i$, com A_i a área em pixels do polígono da vaga. É também o comportamento de generalização ruim entre câmeras que Almeida et al. (2015) observam para classificadores treinados.

5.6 Desempenho computacional

O sistema operou em CPU, em um único *thread*, entre 20,5 e 22,2 FPS por conjunto, com agregado de cerca de 21,2 FPS (12.416 imagens em aproximadamente 585 s somando os nove conjuntos). A medida inclui a decodificação do JPEG e a leitura do XML. A queda para 20,5 a 20,7 FPS na PUCPR é coerente com o maior número de vagas por imagem (100 máscaras por quadro), e a estabilidade do restante mostra que o custo é dominado pelas operações lineares sobre os 1280×720 pixels, como se espera de um pipeline sem etapa iterativa. O comportamento temporal é, portanto, previsível, o que atende ao requisito de tempo real determinístico.

6 Conclusões e trabalhos futuros

Foi implementado e avaliado um detector de vagas baseado apenas em PDI clássico. A calibração em UFPR04 fixou $\tau = 900$ (acurácia de 85,81% e F1 médio de 82,37%) e o

benchmark em 12.416 imagens e 719.528 decisões deu acurácia global de 74,89% e F1 de 70,47%, a cerca de 21,2 FPS em CPU. O método funciona bem quando a geometria da câmera favorece a escala da vaga, como no UFPR04 (87,42%), e executa em tempo real sem GPU, o que sustenta a hipótese de viabilidade para *edge computing*. Não funcionou como regra única para câmeras distantes: com um limiar absoluto, a PUCPR teve *recall* abaixo de 53%. Também há erros residuais por sombras duras, reflexos em piso molhado e textura do piso (bloquete), que a binarização local não distingue de bordas de veículos. As principais limitações são: calibração em um único pátio, composta por subconjunto do próprio benchmark; ausência de divisão treino/teste e de comparação direta com métodos supervisionados; e poucas imagens na condição chuvosa.

Três melhorias decorrem da análise. (1) **Limiar normalizado pela área**: substituir τ por $\tau_i = \kappa A_i$, com A_i a área do polígono, de modo a corrigir a perda de sensibilidade na PUCPR. (2) **Supressão de sombras em HSV**: usar os canais V e S para distinguir sombra, que reduz V sem alterar muito a crominância, de carroceria, antes da binarização. (3) **Integração temporal**: como o estado de uma vaga muda lentamente, exigir confirmação por vários quadros consecutivos eliminaria alarmes transitórios causados por pedestres ou nuvens.

Referências

ALMEIDA, P. R. L. d. et al. PKLot – a robust dataset for parking lot classification. *Expert Systems with Applications*, v. 42, n. 11, p. 4937–4949, 2015. Citado 6 vezes nas páginas 2, 3, 4, 7, 8 e 10.

AMATO, G. et al. Deep learning for decentralized parking lot occupancy detection. *Expert Systems with Applications*, v. 72, p. 327–334, 2017. Citado na página 2.

BRADLEY, D.; ROTH, G. Adaptive thresholding using the integral image. *Journal of Graphics Tools*, v. 12, n. 2, p. 13–21, 2007. Citado na página 3.

BRADSKI, G. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000. Citado na página 4.

FUNCK, S.; MöHLER, N.; OERTEL, W. Determining car-park occupancy from single images. In: *IEEE Intelligent Vehicles Symposium*. [S.l.: s.n.], 2004. p. 325–328. Citado na página 4.

GONZALEZ, R. C.; WOODS, R. E. *Processamento digital de imagens*. 3. ed. São Paulo: Pearson Prentice Hall, 2010. Citado na página 3.

HARALICK, R. M.; STERNBERG, S. R.; ZHUANG, X. Image analysis using mathematical morphology. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 9, n. 4, p. 532–550, 1987. Citado na página 3.

HUANG, C.-C.; WANG, S.-J. A hierarchical Bayesian generation framework for vacant parking space detection. *IEEE Transactions on Circuits and Systems for Video Technology*, v. 20, n. 12, p. 1770–1785, 2010. Citado na página 3.

ICHIHASHI, H. et al. Vacant parking space detector for outdoor parking lot by using surveillance camera and FCM classifier. In: *IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*. [S.l.: s.n.], 2009. p. 127–134. Citado na página 3.

KAPOULKINE, A. *pugixml: light-weight, simple and fast XML parser for C++*. 2024. Disponível em: <https://pugixml.org>. Acesso em: out. 2026. Citado na página 4.

SASTRE, R. et al. Computer algebra algorithms applied to computer vision in a parking management system. In: *IEEE International Symposium on Industrial Electronics (ISIE)*. [S.l.: s.n.], 2007. p. 1675–1680. Citado na página 3.

WOLFF, J. et al. Parking monitor system based on magnetic field sensor. In: *IEEE Intelligent Transportation Systems Conference (ITSC)*. [S.l.: s.n.], 2006. p. 1275–1279. Citado na página 2.

APÊNDICE A – Código-fonte principal

Código-fonte em C++20 do projeto, lido das pastas `include/` e `src/`.

ParkingPipeline.hpp

```cpp
#pragma once

#include <vector>
#include <opencv2/opencv.hpp>
#include "DatasetParser.hpp"

using namespace std;
using namespace cv;

class ParkingPipeline {
private:
    int limiarTau; // Limiar de pixels brancos para decidir se está ocupada

public:
    // Construtor: define o valor inicial de corte (tau)
    ParkingPipeline(int limiar = 700);

    // Permite calibrar o limiar tau em tempo de execução
    void definirLimiar(int novoLimiar);
    int obterLimiar() const;

    // Aplica o pré-processamento de PDI na imagem inteira
    Mat processarImagemPDI(const Mat& entrada);

    // Avalia o estado de cada vaga sobre a imagem binarizada
    void classificarVagas(const Mat& imagemBinarizada, vector<Vaga>& vagas);

    // Desenha as caixas, textos e placar na imagem final
    void renderizarVisualizacao(Mat& imagemOriginal, const vector<Vaga>& vagas);
};
```

ParkingPipeline.cpp

```cpp
#include "ParkingPipeline.hpp"

ParkingPipeline::ParkingPipeline(int limiar) : limiarTau(limiar) {}

void ParkingPipeline::definirLimiar(int novoLimiar) {
    limiarTau = novoLimiar;
}

int ParkingPipeline::obterLimiar() const {
    return limiarTau;
}

Mat ParkingPipeline::processarImagemPDI(const Mat& entrada) {
    Mat cinza, suave, binarizada, dilatada;

    // 1. PDI: Conversão de cor (BGR para Escala de Cinza)
    cvtColor(entrada, cinza, COLOR_BGR2GRAY);

    // 2. PDI: Filtragem Espacial (Filtro Gaussiano 3x3 para atenuar ruído)
    GaussianBlur(cinza, suave, Size(3, 3), 1);

    // 3. PDI: Segmentação por Limiarização Adaptativa (Tolerante a sombras)
    adaptiveThreshold(suave, binarizada, 255,
                      ADAPTIVE_THRESH_GAUSSIAN_C,
                      THRESH_BINARY_INV, 25, 16);

    // 4. PDI: Morfologia Matemática (Dilatação para conectar arestas do veículo)
    Mat elementoEstruturante = getStructuringElement(MORPH_RECT, Size(3, 3));
    dilate(binarizada, dilatada, elementoEstruturante);

    return dilatada;
}

void ParkingPipeline::classificarVagas(const Mat& imagemBinarizada, vector<Vaga>& vagas) {
    for (auto& vaga : vagas) {
        // Valida se a ROI está dentro dos limites da imagem
        Rect roi = vaga.caixaDelimitadora & Rect(0, 0, imagemBinarizada.cols, imagemBinarizada.rows)
    ;
        if (roi.width <= 0 || roi.height <= 0) continue;

        // Recorte da ROI na imagem binarizada
        Mat corteVaga = imagemBinarizada(roi);

        // Cria uma máscara poligonal exata da vaga (elimina ruído dos cantos fora do polígono)
        Mat mascara = Mat::zeros(corteVaga.size(), CV_8UC1);
        vector<Point> pontosLocais;
        for (const auto& pt : vaga.contorno) {
            pontosLocais.push_back(Point(pt.x - roi.x, pt.y - roi.y));
        }
        fillConvexPoly(mascara, pontosLocais, Scalar(255));

        // Aplica a máscara e conta apenas os pixels de dentro da vaga
        Mat vagaIsolada;
        bitwise_and(corteVaga, corteVaga, vagaIsolada, mascara);

        // VC: Extração de característica (densidade de pixels brancos)
        vaga.pixelsBrancos = countNonZero(vagaIsolada);

        // VC: Tomada de decisão por limiar tau
        vaga.ocupadaPredita = (vaga.pixelsBrancos >= limiarTau);
    }
}

void ParkingPipeline::renderizarVisualizacao(Mat& imagemOriginal, const vector<Vaga>& vagas) {
    int livres = 0;
    int ocupadas = 0;

    for (const auto& vaga : vagas) {
        Scalar cor;
```

```
        if (vaga.ocupadaPredita) {
            cor = Scalar(0, 0, 255); // Vermelho: Ocupada
            ocupadas++;
        } else {
            cor = Scalar(0, 255, 0); // Verde: Livre
            livres++;
        }

        // Desenha o polígono da vaga
        polylines(imagemOriginal, vaga.contorno, true, cor, 2);

        // Exibe o ID e os pixels detectados
        if (!vaga.contorno.empty()) {
            string info = to_string(vaga.id) + " (" + to_string(vaga.pixelsBrancos) + ")";
            putText(imagemOriginal, info, vaga.contorno[0],
                    FONT_HERSHEY_PLAIN, 0.9, Scalar(255, 255, 255), 1);
        }
    }

    // Desenha o Placar superior
    string textoPlacar = "Vagas Livres: " + to_string(livres) + " / Total: " + to_string(vagas.size
     ());
    rectangle(imagemOriginal, Point(20, 20), Point(420, 65), Scalar(0, 0, 0), FILLED);
    putText(imagemOriginal, textoPlacar, Point(30, 52), FONT_HERSHEY_SIMPLEX, 0.7, Scalar(0, 255,
     255), 2);
}
```

DatasetParser.hpp

```
#pragma once

#include <vector>
#include <string>
#include <opencv2/opencv.hpp>

using namespace std;
using namespace cv;

// Estrutura que representa uma vaga individual
struct Vaga {
  int id;
  bool ocupadaReal;          // Rótulo do dataset (0 = Livre, 1 = Ocupada)
  bool ocupadaPredita = false; // Predição do nosso algoritmo de PDI
  int pixelsBrancos = 0;     // Quantidade de pixels detectados na vaga
  vector<Point> contorno;    // 4 vértices do polígono da vaga
  Rect caixaDelimitadora;    // Bounding Box para recorte (ROI)
};

class DatasetParser {
public:
  // Carrega todas as vagas a partir do XML da PKLot
  static vector<Vaga> carregarVagasXML(const string& caminhoXML);
};
```

DatasetParser.cpp

```
#include "DatasetParser.hpp"
#include <iostream>
#include <pugixml.hpp>

vector<Vaga> DatasetParser::carregarVagasXML(const string& caminhoXML) {
    vector<Vaga> vagas;
    pugi::xml_document doc;

    pugi::xml_parse_result resultado = doc.load_file(caminhoXML.c_str());
```

```
    if (!resultado) {
        cerr << "[ERRO] Nao foi possivel carregar o XML: " << caminhoXML << endl;
        cerr << "Descricao do erro: " << resultado.description() << endl;
        return vagas;
    }

    pugi::xml_node parkingNode = doc.child("parking");
    for (pugi::xml_node spaceNode = parkingNode.child("space"); spaceNode; spaceNode = spaceNode.
     next_sibling("space")) {
        Vaga v;
        v.id = spaceNode.attribute("id").as_int();
        v.ocupadaReal = (spaceNode.attribute("occupied").as_int() == 1);

        pugi::xml_node contourNode = spaceNode.child("contour");
        for (pugi::xml_node pointNode = contourNode.child("point"); pointNode; pointNode = pointNode
     .next_sibling("point")) {
            int x = pointNode.attribute("x").as_int();
            int y = pointNode.attribute("y").as_int();
            v.contorno.push_back(Point(x, y));
        }

        // Calcula a Bounding Box manualmente (sem precisar da função boundingRect)
        if (!v.contorno.empty()) {
            int minX = v.contorno[0].x;
            int maxX = v.contorno[0].x;
            int minY = v.contorno[0].y;
            int maxY = v.contorno[0].y;

            for (const auto& p : v.contorno) {
                if (p.x < minX) minX = p.x;
                if (p.x > maxX) maxX = p.x;
                if (p.y < minY) minY = p.y;
                if (p.y > maxY) maxY = p.y;
            }

            v.caixaDelimitadora = Rect(minX, minY, maxX - minX, maxY - minY);
        }

        vagas.push_back(v);
    }

    return vagas;
}
```

Evaluator.hpp

```
#pragma once

#include <string>
#include <vector>
#include "ParkingPipeline.hpp"

using namespace std;

struct MetricasClima {
    string clima;
    int totalImagens = 0;
    int totalVagas = 0;
    int vp = 0;
    int vn = 0;
    int fp = 0;
    int fn = 0;
    double tempoTotalSegundos = 0.0;

    double acuracia() const {
        return (totalVagas > 0) ? (double) (vp + vn) / totalVagas * 100.0 : 0.0;
    }
```

```
22
23      double precisao() const {
24          return (vp + fp > 0) ? (double) vp / (vp + fp) * 100.0 : 0.0;
25      }
26
27      double recall() const {
28          return (vp + fn > 0) ? (double) vp / (vp + fn) * 100.0 : 0.0;
29      }
30
31      double f1Score() const {
32          double p = precisao();
33          double r = recall();
34          return (p + r > 0.0) ? 2.0 * (p * r) / (p + r) : 0.0;
35      }
36
37      double taxaErro() const {
38          return (totalVagas > 0) ? (double) (fp + fn) / totalVagas * 100.0 : 0.0;
39      }
40
41      double fps() const {
42          return (tempoTotalSegundos > 0.0) ? (double) totalImagens / tempoTotalSegundos : 0.0;
43      }
44  };
45
46  class Evaluator {
47  public:
48      static MetricasClima avaliarClima(const string &pastaClima,
49                                        const string &nomeClima,
50                                        ParkingPipeline &pipeline,
51                                        int maxImagens = -1);
52
53      static void imprimirTabelaComparativa(const vector<MetricasClima> &resultados);
54
55      static void salvarHistoricoCSV(const string &caminhoArquivo, int limiarTau,
56                                     const vector<MetricasClima> &resultados);
57  };
```

Evaluator.cpp

```
1   #include "Evaluator.hpp"
2   #include "DatasetParser.hpp"
3   #include <iostream>
4   #include <filesystem>
5   #include <chrono>
6   #include <iomanip>
7   #include <fstream>
8   #include <ctime>
9
10  namespace fs = std::filesystem;
11
12  MetricasClima Evaluator::avaliarClima(const string &pastaClima,
13                                        const string &nomeClima,
14                                        ParkingPipeline &pipeline,
15                                        int maxImagens) {
16      MetricasClima m;
17      m.clima = nomeClima;
18
19      if (!fs::exists(pastaClima)) {
20          cerr << "[AVISO] Pasta nao encontrada: " << pastaClima << endl;
21          return m;
22      }
23
24      cout << "\n======================================================" << endl;
25      cout << "Iniciando avaliacao para o clima: " << nomeClima << endl;
26      cout << "Pasta: " << pastaClima << endl;
27      cout << "======================================================" << endl;
28
29      auto inicioTempo = chrono::high_resolution_clock::now();
30
31      // Itera recursivamente por todas as pastas de data dentro do clima
32      for (const auto &entry: fs::recursive_directory_iterator(pastaClima)) {
33          if (entry.is_regular_file() && entry.path().extension() == ".jpg") {
34              string caminhoJPG = entry.path().string();
```

```
35
36              // Procura o XML de mesmo nome
37              fs::path caminhoXMLPath = entry.path();
38              caminhoXMLPath.replace_extension(".xml");
39
40              if (!fs::exists(caminhoXMLPath)) {
41                  continue;
42              }
43
44              // 1. Carrega imagem e XML
45              Mat frame = imread(caminhoJPG);
46              if (frame.empty()) continue;
47
48              vector<Vaga> vagas = DatasetParser::carregarVagasXML(caminhoXMLPath.string());
49              if (vagas.empty()) continue;
50
51              // 2. Executa PDI e Decisão
52              Mat binaria = pipeline.processarImagemPDI(frame);
53              pipeline.classificarVagas(binaria, vagas);
54
55              // 3. Atualiza matriz de confusão
56              for (const auto &v: vagas) {
57                  m.totalVagas++;
58                  if (v.ocupadaReal && v.ocupadaPredita) {
59                      m.vp++;
60                  } else if (!v.ocupadaReal && !v.ocupadaPredita) {
61                      m.vn++;
62                  } else if (!v.ocupadaReal && v.ocupadaPredita) {
63                      m.fp++; // Falso alarme
64                  } else if (v.ocupadaReal && !v.ocupadaPredita) {
65                      m.fn++; // Perdeu o carro
66                  }
67              }
68
69              m.totalImagens++;
70
71              // Feedback de progresso no console a cada 100 imagens
72              if (m.totalImagens % 100 == 0) {
73                  cout << "  Processadas " << m.totalImagens << " imagens (" << m.totalVagas << " vagas analisadas)
      ..." <<
74                          endl;
75              }
76
77              // Limite opcional para testes rápidos
78              if (maxImagens > 0 && m.totalImagens >= maxImagens) {
79                  break;
80              }
81          }
82      }
83
84      auto fimTempo = chrono::high_resolution_clock::now();
85      chrono::duration<double> duracao = fimTempo - inicioTempo;
86      m.tempoTotalSegundos = duracao.count();
87
88      cout << "Concluido clima " << nomeClima << "! Total: " << m.totalImagens
89              << " imagens em " << fixed << setprecision(2) << m.tempoTotalSegundos
90              << "s (" << m.fps() << " FPS)." << endl;
91
92      return m;
93  }
94
95  void Evaluator::imprimirTabelaComparativa(const vector<MetricasClima> &resultados) {
96      cout << "\n\n";
97      cout <<
98              "
        ==========================================================================================================\n";
99      cout <<
100             "                                TABELA COMPARATIVA DE RESULTADOS (PKLot)
          \n";
101     cout <<
102             "
        ==========================================================================================================\n";
103     cout << left << setw(10) << "Clima"
104             << setw(10) << "Imagens"
105             << setw(10) << "Vagas"
106             << setw(8) << "VP"
107             << setw(8) << "VN"
108             << setw(8) << "FP"
109             << setw(8) << "FN"
110             << setw(12) << "Acuracia"
111             << setw(12) << "Precisao"
112             << setw(10) << "Recall"
113             << setw(10) << "F1-Score"
```

```
            << setw(8) << "FPS" << "\n";
    cout <<
          "
     ----------------------------------------------------------------------------------------------------------\n";

    for (const auto &r: resultados) {
        cout << left << setw(10) << r.clima
                << setw(10) << r.totalImagens
                << setw(10) << r.totalVagas
                << setw(8) << r.vp
                << setw(8) << r.vn
                << setw(8) << r.fp
                << setw(8) << r.fn
                << fixed << setprecision(2)
                << setw(11) << to_string(r.acuracia()).substr(0, 5) + "%"
                << setw(11) << to_string(r.precisao()).substr(0, 5) + "%"
                << setw(10) << to_string(r.recall()).substr(0, 5) + "%"
                << setw(10) << to_string(r.f1Score()).substr(0, 5) + "%"
                << setw(8) << setprecision(1) << r.fps() << "\n";
    }
    cout <<
          "
     ==========================================================================================================\n";
}

void Evaluator::salvarHistoricoCSV(const string &caminhoArquivo,
                                   int limiarTau,
                                   const vector<MetricasClima> &resultados) {
    bool arquivoJaExiste = fs::exists(caminhoArquivo);
    ofstream arquivo(caminhoArquivo, ios::app); // Abre em modo append (não sobrescreve)

    if (!arquivo.is_open()) {
        cerr << "[ERRO] Nao foi possivel abrir o arquivo para gravacao: " << caminhoArquivo << endl;
        return;
    }

    // Se o arquivo for criado agora, escreve o cabeçalho separado por ponto e vírgula (formato Excel Brasil)
    if (!arquivoJaExiste) {
        arquivo <<
                "DataHora;Limiar_Tau;Clima;Imagens;Vagas;VP;VN;FP;FN;Acuracia_Porc;Precisao_Porc;Recall_Porc;
     F1_Score_Porc;Taxa_Erro_Porc;FPS\n";
    }

    // Obtém a data e hora do experimento
    auto agora = chrono::system_clock::to_time_t(chrono::system_clock::now());
    char bufferData[30];
    tm ltm;
    localtime_s(&ltm, &agora); // Função segura no Windows MSVC
    strftime(bufferData, sizeof(bufferData), "%Y-%m-%d %H:%M:%S", &ltm);

    // Grava as linhas de cada clima com o limiar correspondente
    for (const auto &r: resultados) {
        arquivo << bufferData << ";"
                << limiarTau << ";"
                << r.clima << ";"
                << r.totalImagens << ";"
                << r.totalVagas << ";"
                << r.vp << ";"
                << r.vn << ";"
                << r.fp << ";"
                << r.fn << ";"
                << fixed << setprecision(2)
                << r.acuracia() << ";"
                << r.precisao() << ";"
                << r.recall() << ";"
                << r.f1Score() << ";"
                << r.taxaErro() << ";"
                << setprecision(1) << r.fps() << "\n";
    }

    arquivo.close();
    cout << "\n[HISTORICO] Resultados salvos com sucesso em: " << caminhoArquivo << endl;
}
```

main.cpp

```
#include <iostream>
#include <vector>
#include <string>
```

```
4  #include <filesystem>
5  #include <opencv2/opencv.hpp>
6  #include "ParkingPipeline.hpp"
7  #include "DatasetParser.hpp"
8  #include "Evaluator.hpp"
9
10 using namespace std;
11 namespace fs = std::filesystem;
12
13 // Raiz da base PKLot, relativa ao diretório de execução (cmake-build-*)
14 static const string CAMINHO_BASE_PKLOT = "../data/PKLot/PKLot/";
15 static const int TAU_OTIMO = 900;
16
17 // ============================================================================
18 // MODO 1 (padrão): avaliação global do dataset completo - comportamento original
19 // ============================================================================
20 static int executarBenchmark() {
21     cout << "======================================================" << endl;
22     cout << "   AVALIACAO GLOBAL DO DATASET COMPLETO (PKLot)      " << endl;
23     cout << "======================================================" << endl;
24
25     // limiar ótimo comprovado nos experimentos preliminares
26     int tauOtimo = TAU_OTIMO;
27     ParkingPipeline pipeline(tauOtimo);
28
29     // O valor -1 desativa qualquer limite e processa TODAS as fotos
30     int limitePorClima = -1;
31
32     // Lista dos 3 pátios da base PKLot
33     vector<string> patios = {"UFPR04", "UFPR05", "PUCPR"};
34     vector<string> climas = {"Sunny", "Cloudy", "Rainy"};
35
36     vector<MetricasClima> resultadosGlobais;
37
38     string caminhoBasePKLot = CAMINHO_BASE_PKLOT;
39
40     // Percorre cada pátio e cada clima
41     for (const auto &patio: patios) {
42         cout << "\n>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>" << endl;
43         cout << "Iniciando processamento do patio: " << patio << endl;
44         cout << ">>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>" << endl;
45
46         for (const auto &clima: climas) {
47             string pastaClima = caminhoBasePKLot + patio + "/" + clima;
48             string identificador = patio + "_" + clima;
49
50             MetricasClima m = Evaluator::avaliarClima(pastaClima, identificador, pipeline,
    limitePorClima);
51             resultadosGlobais.push_back(m);
52         }
53     }
54
55     // 1. Imprime a tabela comparativa geral no console
56     Evaluator::imprimirTabelaComparativa(resultadosGlobais);
57
58     // 2. Salva o resultado massivo no histórico CSV
59     Evaluator::salvarHistoricoCSV("../historico_final_tau_900.csv", tauOtimo, resultadosGlobais);
60
61     cout << "\n[FINALIZADO] Todas as imagens foram processadas e registradas com sucesso!" << endl;
62     return 0;
63 }
64
65 // ============================================================================
66 // MODO 2 (--galeria): mosaico 3x3 (pátio x clima) + mosaico das imagens binarizadas
67 // ============================================================================
68 struct AmostraGaleria {
69     string patio;
70     string clima;
71     string caminhoSemExtensao; // relativo a CAMINHO_BASE_PKLOT, sem .jpg/.xml
72 };
```

```
73
74  // Linhas = pátios (UFPR04, UFPR05, PUCPR); colunas = clima (Sunny, Cloudy, Rainy)
75  static const vector<AmostraGaleria> AMOSTRAS = {
76      {"UFPR04", "Sunny", "UFPR04/Sunny/2013-01-29/2013-01-29_20_26_23"},
77      {"UFPR04", "Cloudy", "UFPR04/Cloudy/2013-01-29/2013-01-29_16_21_18"},
78      {"UFPR04", "Rainy", "UFPR04/Rainy/2013-01-21/2013-01-21_10_00_05"},
79
80      {"UFPR05", "Sunny", "UFPR05/Sunny/2013-14-16/2013-04-16_07_50_02"},
81      {"UFPR05", "Cloudy", "UFPR05/Cloudy/2013-03-13/2013-03-13_08_10_02"},
82      {"UFPR05", "Rainy", "UFPR05/Rainy/2013-02-26/2013-02-26_13_19_33"},
83
84      {"PUCPR", "Sunny", "PUCPR/Sunny/2012-11-20/2012-11-20_15_09_41"},
85      {"PUCPR", "Cloudy", "PUCPR/Cloudy/2012-09-28/2012-09-28_07_51_02"},
86      {"PUCPR", "Rainy", "PUCPR/Rainy/2012-11-10/2012-11-10_08_12_43"},
87  };
88
89  // Tamanho de cada célula do mosaico (proporção 16:9 das imagens 1280x720)
90  static const int LARG_CELULA = 640;
91  static const int ALT_CELULA = 360;
92
93  // Localiza o .jpg da amostra. Se o caminho exato não existir (ex.: nome de pasta
94  // de data diferente), procura o arquivo pelo nome dentro de pátio/clima.
95  static string localizarJPG(const AmostraGaleria &a) {
96      string direto = CAMINHO_BASE_PKLOT + a.caminhoSemExtensao + ".jpg";
97      if (fs::exists(direto)) return direto;
98
99      string nomeArquivo = fs::path(a.caminhoSemExtensao).filename().string() + ".jpg";
100     string pastaBusca = CAMINHO_BASE_PKLOT + a.patio + "/" + a.clima;
101     if (fs::exists(pastaBusca)) {
102         for (const auto &entry: fs::recursive_directory_iterator(pastaBusca)) {
103             if (entry.is_regular_file() && entry.path().filename().string() == nomeArquivo) {
104                 cerr << "[AVISO] Caminho exato nao encontrado; usando: " << entry.path().string() <<
      endl;
105                 return entry.path().string();
106             }
107         }
108     }
109     return "";
110 }
111
112 // Célula cinza indicando que a amostra não pôde ser carregada
113 static Mat celulaIndisponivel(const AmostraGaleria &a) {
114     Mat c(ALT_CELULA, LARG_CELULA, CV_8UC3, Scalar(60, 60, 60));
115     putText(c, a.patio + " - " + a.clima, Point(20, 150), FONT_HERSHEY_SIMPLEX, 0.8, Scalar(0, 255,
       255), 2);
116     putText(c, "Amostra nao encontrada", Point(20, 190), FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255, 255,
        255), 2);
117     return c;
118 }
119
120 // Desenha as caixas (verde = livre, vermelho = ocupada, segundo a PREDIÇÃO) e o placar.
121 // 'escala' converte as coordenadas da imagem original para a célula redimensionada.
122 static void desenharCaixasEPlacar(Mat &celula, const vector<Vaga> &vagas, double escala,
123                                   const string &titulo) {
124     int ocupadasPred = 0;
125     int ocupadasReal = 0;
126
127     for (const auto &v: vagas) {
128         if (v.ocupadaPredita) ocupadasPred++;
129         if (v.ocupadaReal) ocupadasReal++;
130
131         vector<Point> poligono;
132         for (const auto &p: v.contorno) {
133             poligono.push_back(Point(cvRound(p.x * escala), cvRound(p.y * escala)));
134         }
135         Scalar cor = v.ocupadaPredita ? Scalar(0, 0, 255) : Scalar(0, 255, 0);
136         polylines(celula, poligono, true, cor, 2);
137     }
138
139     int total = (int) vagas.size();
```

```
140     int livresPred = total - ocupadasPred;
141
142     // Placar em UMA linha, em faixa fina na base da célula
143     const int ALTURA_FAIXA = 20;
144     const double FONTE = 0.5;
145     rectangle(celula, Point(0, celula.rows - ALTURA_FAIXA), Point(celula.cols, celula.rows),
146               Scalar(0, 0, 0), FILLED);
147
148     int x = 6;
149     int y = celula.rows - 6;
150     auto escrever = [&](const string &texto, const Scalar &cor) {
151         putText(celula, texto, Point(x, y), FONT_HERSHEY_SIMPLEX, FONTE, cor, 1, LINE_AA);
152         int base = 0;
153         x += getTextSize(texto, FONT_HERSHEY_SIMPLEX, FONTE, 1, &base).width;
154     };
155     escrever(titulo, Scalar(0, 255, 255));
156     escrever("  |  Vagas: " + to_string(total), Scalar(255, 255, 255));
157     escrever("  Ocupadas: " + to_string(ocupadasPred), Scalar(80, 80, 255));
158     escrever("  Livres: " + to_string(livresPred), Scalar(80, 255, 80));
159     escrever("  |  Gabarito: " + to_string(ocupadasReal), Scalar(180, 180, 180));
160
161     rectangle(celula, Point(0, 0), Point(celula.cols - 1, celula.rows - 1), Scalar(255, 255, 255),
        1);
162 }
163
164 // Processa uma amostra e devolve (célula colorida com caixas, célula binarizada com caixas)
165 static pair<Mat, Mat> processarAmostra(const AmostraGaleria &a, ParkingPipeline &pipeline) {
166     string caminhoJPG = localizarJPG(a);
167     if (caminhoJPG.empty()) {
168         cerr << "[ERRO] JPG nao encontrado para: " << a.caminhoSemExtensao << endl;
169         return {celulaIndisponivel(a), celulaIndisponivel(a)};
170     }
171
172     fs::path caminhoXML = fs::path(caminhoJPG).replace_extension(".xml");
173     Mat frame = imread(caminhoJPG);
174     vector<Vaga> vagas = DatasetParser::carregarVagasXML(caminhoXML.string());
175     if (frame.empty() || vagas.empty()) {
176         cerr << "[ERRO] Falha ao carregar imagem/XML: " << caminhoJPG << endl;
177         return {celulaIndisponivel(a), celulaIndisponivel(a)};
178     }
179
180     // Mesmo pipeline do benchmark: PDI global + classificação por vaga
181     Mat binaria = pipeline.processarImagemPDI(frame);
182     pipeline.classificarVagas(binaria, vagas);
183
184     double escala = (double) LARG_CELULA / frame.cols;
185     string titulo = a.patio + " - " + a.clima;
186
187     Mat celulaCor, celulaBin, celulaBinBGR;
188     resize(frame, celulaCor, Size(LARG_CELULA, ALT_CELULA), 0, 0, INTER_AREA);
189     resize(binaria, celulaBin, Size(LARG_CELULA, ALT_CELULA), 0, 0, INTER_AREA);
190     cvtColor(celulaBin, celulaBinBGR, COLOR_GRAY2BGR);
191
192     desenharCaixasEPlacar(celulaCor, vagas, escala, titulo);
193     desenharCaixasEPlacar(celulaBinBGR, vagas, escala, titulo);
194
195     return {celulaCor, celulaBinBGR};
196 }
197
198 static int executarGaleria(bool mostrarJanelas) {
199     cout << "====================================================" << endl;
200     cout << "   GALERIA 3x3 DE AMOSTRAS (PKLot) - tau = " << TAU_OTIMO << endl;
201     cout << "====================================================" << endl;
202
203     ParkingPipeline pipeline(TAU_OTIMO);
204
205     Mat mosaicoCor(3 * ALT_CELULA, 3 * LARG_CELULA, CV_8UC3, Scalar(0, 0, 0));
206     Mat mosaicoBin(3 * ALT_CELULA, 3 * LARG_CELULA, CV_8UC3, Scalar(0, 0, 0));
207
208     for (size_t i = 0; i < AMOSTRAS.size(); i++) {
```

```
209         int linha = (int) i / 3;
210         int coluna = (int) i % 3;
211         cout << "Processando " << AMOSTRAS[i].patio << " / " << AMOSTRAS[i].clima << "..." << endl;
212 
213         pair<Mat, Mat> celulas = processarAmostra(AMOSTRAS[i], pipeline);
214         Rect destino(coluna * LARG_CELULA, linha * ALT_CELULA, LARG_CELULA, ALT_CELULA);
215         celulas.first.copyTo(mosaicoCor(destino));
216         celulas.second.copyTo(mosaicoBin(destino));
217     }
218 
219     // Salva automaticamente os mosaicos (usados como figuras do artigo)
220     const string arquivoCor = "../graficos/galeria_exemplos.png";
221     const string arquivoBin = "../graficos/galeria_binarizada.png";
222     fs::create_directories("../graficos");
223     bool okCor = imwrite(arquivoCor, mosaicoCor);
224     bool okBin = imwrite(arquivoBin, mosaicoBin);
225     if (okCor && okBin) {
226         cout << "\n[SALVO] " << arquivoCor << "\n[SALVO] " << arquivoBin << endl;
227     } else {
228         cerr << "[ERRO] Nao foi possivel salvar os mosaicos em ../graficos/" << endl;
229     }
230 
231     if (!mostrarJanelas) {
232         return (okCor && okBin) ? 0 : 1;
233     }
234 
235     const string janelaCor = "Galeria PKLot - Deteccao (verde = livre, vermelho = ocupada)";
236     const string janelaBin = "Galeria PKLot - Imagens binarizadas (adaptiveThreshold + dilatacao)";
237 
238     // Janelas redimensionáveis; o mosaico interno mantém 1920x1080 para salvar em boa resolução
239     namedWindow(janelaCor, WINDOW_NORMAL);
240     namedWindow(janelaBin, WINDOW_NORMAL);
241     resizeWindow(janelaCor, 1440, 810);
242     resizeWindow(janelaBin, 1440, 810);
243     moveWindow(janelaCor, 20, 20);
244     moveWindow(janelaBin, 80, 80);
245     imshow(janelaCor, mosaicoCor);
246     imshow(janelaBin, mosaicoBin);
247 
248     cout << "\n[ATALHOS] 's' = salvar novamente | 'q' ou ESC = sair\n" << endl;
249 
250     while (true) {
251         int tecla = waitKey(50) & 0xFF;
252         if (tecla == 27 || tecla == 'q') break;
253 
254         if (tecla == 's') {
255             imwrite(arquivoCor, mosaicoCor);
256             imwrite(arquivoBin, mosaicoBin);
257             cout << "[SALVO] " << arquivoCor << " e " << arquivoBin << endl;
258         }
259 
260         // Encerra se o usuário fechar qualquer uma das janelas no botão X
261         if (getWindowProperty(janelaCor, WND_PROP_VISIBLE) < 1 ||
262             getWindowProperty(janelaBin, WND_PROP_VISIBLE) < 1) {
263             break;
264         }
265     }
266 
267     destroyAllWindows();
268     return 0;
269 }
270 
271 // ============================================================================
272 int main(int argc, char **argv) {
273     // Sem argumentos: comportamento original (benchmark completo).
274     // --galeria: gera e salva os mosaicos 3x3 (deteccao + binarizadas) e abre as janelas.
275     // --galeria --sem-janelas: apenas gera e salva os PNGs (sem abrir janelas).
276     bool galeria = false;
277     bool semJanelas = false;
278 
```

```
279     for (int i = 1; i < argc; i++) {
280         string opcao = argv[i];
281         if (opcao == "--galeria") {
282             galeria = true;
283         } else if (opcao == "--sem-janelas") {
284             semJanelas = true;
285         } else {
286             cout << "Uso: " << argv[0] << " [--galeria] [--sem-janelas]\n"
287                     << "  (sem argumentos)         avaliacao global do dataset completo (benchmark)\
    n"
288                     << "  --galeria                salva os mosaicos 3x3 em ../graficos/ e exibe as
    janelas\n"
289                     << "  --galeria --sem-janelas  apenas salva os mosaicos, sem abrir janelas" <<
    endl;
290             return (opcao == "--ajuda" || opcao == "--help") ? 0 : 1;
291         }
292     }
293 
294     if (galeria) {
295         return executarGaleria(!semJanelas);
296     }
297     if (semJanelas) {
298         cout << "[AVISO] --sem-janelas so tem efeito junto com --galeria." << endl;
299         return 1;
300     }
301     return executarBenchmark();
302 }
```